Password Manager Project — Detailed Documentation

Overview

This project is a **RoboForm-inspired password management and credential storage application** built using **Python and Tkinter**. The program allows users to generate secure passwords, evaluate password strength, store credentials securely, simulate login autofill, and store credit card information with validation.

The application demonstrates several core security concepts:

- Secure password generation
- Password strength evaluation
- Encryption of stored credentials
- Secure data storage using JSON
- Input validation for sensitive information
- GUI application design using Tkinter

The application runs locally as a **desktop program** and does not connect to external servers.

System Architecture

The program follows a **modular design**, separating functionality into several Python modules.

```
main.py
│
├── password_generator.py
├── password_checker_connector.py
├── storage.py
├── encryption.py
├── credit_card_utils.py
└── credit_card_storage.py
```

Each module is responsible for a specific part of the system.

Application Pages

The GUI is composed of several **Tkinter frames** that act as pages.

Pages

Page	Purpose
HomePage	Main menu navigation
PasswordGeneratorPage	Generate secure passwords
AutofillPage	Simulated login with password autofill
CreditCardPage	Store and validate credit card data

The application controller (`PasswordManagerApp`) manages navigation between pages.

Feature Documentation

Password Generator

Location

`password_generator.py`

Purpose

Generates strong random passwords using cryptographically secure randomness.

Randomness Source

Passwords are generated using:

secrets.choice()

The `secrets` module is designed for **cryptographic security**, unlike Python's `random` module.

Character Sets

The generator can include:

Character Type	Source
Lowercase	`string.ascii_lowercase`
Uppercase	`string.ascii_uppercase`
Numbers	`string.digits`
Symbols	`string.punctuation`

Advanced Settings

The generator also supports additional options similar to RoboForm.

Exclude Similar Characters

Characters that can be visually confused are removed:

i l I 1 o O 0

This helps prevent mistakes when manually entering passwords.

Hexadecimal Mode

Restricts characters to:

0-9 A-F

Used for generating passwords compatible with systems that only allow hexadecimal characters.

Password Length

Passwords can range from:

1 to 512 characters

The UI uses a **spinbox selector** to adjust length.

Character Requirement Enforcement

When generating a password, the program verifies that the password contains at least one character from each selected category.

Example validation:

if lowercase selected → must contain lowercase
if uppercase selected → must contain uppercase
if digits selected → must contain digits
if symbols selected → must contain symbols

If the password fails these checks, a new password is generated.

Password Strength Scoring

Location

`password_checker_connector.py`

Purpose

Evaluates password strength using multiple heuristics and generates improvement suggestions.

Maximum Score

Maximum theoretical score:

115

Scores may become **negative** if severe weaknesses are detected.

Scoring Categories

Length

Length	Score
<8	-50
8–11	+15
12–15	+20
16+	+25

English Word Detection

The program loads a large dictionary:

lists/words.txt

If a password contains an English word (4+ characters):

Penalty: -10

Common Password Detection

The program compares against:

100k-most-used-passwords-NCSC.txt

If the password is found:

Penalty: -500

Capitalization

Pattern	Score
Lower + Upper	+25
Only one case	-20

Special Characters

Rewards are given based on placement.

Condition	Score
None	0
Edge only	+5
Middle	+15
Multiple middle	+25

Numbers

Numbers in the middle receive higher scores.

Ascending sequences (example `123`) are penalized.

Strength Classification

Final strength categories:

Score	Strength
≤20	Very Weak
21–40	Weak
41–60	Moderate
61–80	Strong
81+	Very Strong

Password Score Visualization

The UI includes a **dynamic progress bar**.

The bar:

- fills based on score
- changes color based on strength

Strength	Color
Very Weak	Red
Weak	Orange
Moderate	Yellow
Strong	Light Green
Very Strong	Dark Green

Suggestions System

If weaknesses are detected, suggestions are displayed on screen.

Example suggestions:

Avoid using english words
Add numbers
Add special characters
Avoid sequences of numbers
Use mixed capitalization

Suggestions are generated during scoring.

Password Storage

Location

`storage.py`

Passwords are stored in **two separate files**:

passwords_plain.json
passwords_encrypted.json

Plaintext Storage

Plaintext storage allows easy viewing and debugging.

Structure example:

```
[
{
"website": "example.com",
"username": "user123",
"password": "SecurePass!42"
}
]
```

Encrypted Storage

Encrypted records use the **cryptography library (Fernet encryption)**.

cryptography.fernet

Encrypted file example:

passwords_encrypted.json

Passwords are encrypted before writing to disk.

Encryption Key

The key is stored in:

secret.key

This key is generated automatically if missing.

Autofill Login Simulation

Location

`AutofillPage` in `main.py`

Purpose

Demonstrates password autofill functionality similar to browser password managers.

Process

1. User enters:

website
username

2. Program searches stored encrypted records
3. If a match is found:

password is decrypted
password field is autofilled

Password Visibility Toggle

Users can toggle password visibility:

Show / Hide

Credit Card Storage

Location

`credit_card_utils.py`
`credit_card_storage.py`

The system validates and stores credit card information.

Credit Card Validation

Validation checks include:

Card Type Detection

Supported cards:

Card	Prefix
Visa	4
Mastercard	51–55, 2221–2720
American Express	34 or 37

Card Length

Card	Length
Visa	13, 16, or 19
Mastercard	16
AmEx	15

Expiration Date

Formats accepted:

MM/YY
MM/YYYY

Expired cards are rejected.

CVC Validation

Card	CVC Length
Visa	3
Mastercard	3
AmEx	4

Credit Card Storage

Like passwords, cards are stored in two files:

credit_cards_plain.json
credit_cards_encrypted.json

Unique Record Identifier

Each stored credit card receives a **unique ID**.

This allows:

- updates to existing cards
- prevention of duplicate entries

Security Considerations

The project demonstrates several important security practices.

Cryptographically Secure Randomness

secrets module

Used for generating passwords.

Encryption

Sensitive data is encrypted using:

Fernet symmetric encryption

from the `cryptography` library.

Input Validation

Strict validation prevents invalid inputs such as:

- invalid credit card numbers
- expired cards
- invalid CVC codes

Separation of Storage

Data is separated into:

plaintext
encrypted

This allows demonstration and testing of encryption.

Dependencies

External dependency:

cryptography

Install using:

pip install cryptography

All other libraries are part of the Python standard library.

Running the Application

Run the program with:

python main.py

Local Files Generated

During execution the program may create:

secret.key
passwords_plain.json
passwords_encrypted.json
credit_cards_plain.json
credit_cards_encrypted.json

These files are used for storing local data.